Una URL que funciona.

Prof: Manuela Larrea Gomez

CUANTIZAR | SERVIR | DESPLEGAR | OBSERVAR

La sesión de hoy: la última con tema nuevo.

- Recap: resolvemos las dudas del formulario de salida (la última ronda)
- Cuantizar: los mismos pesos con menos bits — demo en vivo fp16 vs. int4
- Servir: por qué ChatGPT no se "apaga"; vLLM, latencia p50/p95
- Desplegar: app.py → Space de HF → URL pública, con health check y logs 婶 Lab
- Observar: W&B en producción · la rúbrica de "producción verificable"
- Cierre: la semana que viene, y una despedida

Primero,
sus dudas.

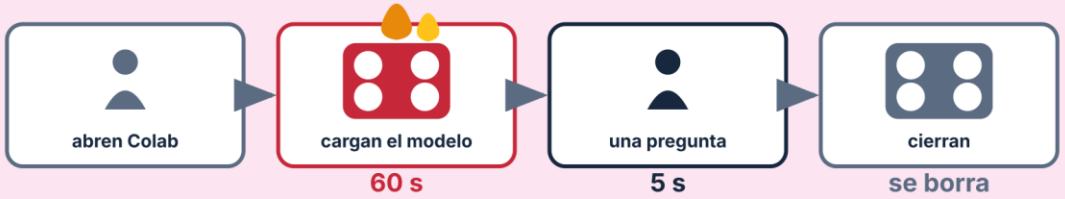
La última ronda. Cuatro temas: cómo hace ChatGPT para responder tan rápido · cómo se usa un modelo en una empresa y cómo se integra · la temperatura y tres dudas sobre el profesor · costos, las reglas de M5 y lo que DevOps ya sabía.

¿Cómo hace ChatGPT para responder tan rápido? El modelo nunca se apaga.

como un restaurante: la cocina ya está prendida

EN SU NOTEBOOK

la cocina se prende para cada cliente

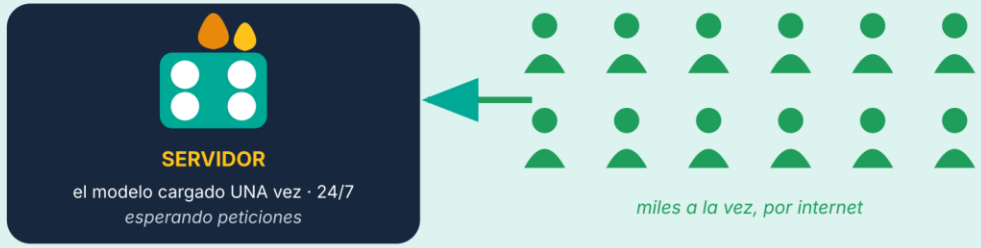


1 cliente a la vez · 1 minuto de espera · 14 GB cargados cada vez

y al cerrar la pestaña, la memoria se vacía

EN PRODUCCIÓN

la cocina está prendida desde la mañana



miles de clientes · segundos de espera · nadie carga nada

usted no "entra" al modelo: le manda un mensaje a un programa que ya lo tiene en memoria

Y para que "segundos" sean segundos aunque entren miles: cuatro trucos

Batching
muchas preguntas, una sola pasada por la GPU

KV cache
lo ya leído no se recalcula: se guarda

Streaming
cada palabra viaja en cuanto existe

Réplicas
muchas cocinas iguales + un balanceador

Su Space de HF es exactamente esto, en pequeño: un servidor que carga su modelo una vez y atiende por una URL.

El modelo no se "abre" cuando usted entra: ya estaba en memoria. Servir = cargar una vez, atender muchas.

batching + KV cache + streaming + réplicas: por eso ChatGPT parece instantáneo con millones de usuarios

¿Cómo se usa un modelo en una empresa? Casi nunca como un chat.

cuatro formas de usarlo, una sola regla, tres capas

A · CHAT
visible pero rara

**un humano escribe,
el modelo responde**

ej.: asistente para asesores

B · AGENTE
el patrón que más crece

**recibe una tarea, usa tools,
devuelve un resultado**

ej.: "revisa el reclamo y propón"

C · BATCH
invisible pero común

**un proceso lo llama
miles de veces, de noche**

ej.: 50 000 fotos cada madrugada

D · EMBEBIDO
nadie sabe que hay un LLM

**dentro de un botón
que ya existía**

ej.: "sugerir respuesta"

← lo que todos imaginan

lo que de verdad corre →

TODAS PASAN POR LO MISMO:
una API

`POST /responder · JSON entra, JSON sale`

nadie "ve" el modelo: ven la API. El modelo se envuelve en una API, y la API se conecta a lo que ya existe.

¿Y cómo se conecta a la empresa? Tres capas, siempre las mismas

INTERFAZ
lo que el usuario toca

chat web · botón en el CRM · bot de Teams · o nada (batch)

la hace el equipo de PRODUCTO

GATEWAY
la API de la empresa

autenticación · límites · logging · costo por usuario · MCP

la hace el equipo de PLATAFORMA

MODELO
lo que ustedes construyeron

servido (vLLM, un Space, o la API de un proveedor) + RAG + tools

la hacen USTEDES: su Space de hoy

la interfaz casi nunca la hace el equipo de IA: la hace producto, contra la API. Por eso la API es el contrato.

Cuatro formas: chat, agente, batch, embebido. Una regla: el modelo se envuelve en una API y la API se conecta a lo que ya existe.

la forma más común no es la que se ve (el chat) sino la que no se ve (batch y embebido)

La temperatura, y tres dudas sobre el profesor

"¿cómo manejan la temperatura?" · "¿cualquier modelo puede ser profesor?" · "¿se sabe de qué se destiló?" · "¿cómo no incumplir políticas?"

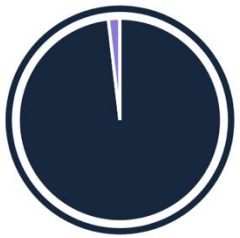
1 · LA TEMPERATURA ES UN DIAL SOBRE LA RULETA

el modelo no elige la siguiente palabra: la sortea. T decide qué tan abierta es la ruleta.



← *cerrada: determinista*

abierta: variada →



T = 0.2

casi siempre "roya"

hechos · campos · citas · la demo



T = 1

lo que aprendió

equilibrio



T = 2

aparecen los parecidos

creatividad · destilar (S14)

roya · minador · hongo · sana

```
p(palabra) ∝ exp( logit / T )
```

baja para hechos · alta para variedad o para ablandar al profesor

2 · TRES DUDAS SOBRE EL PROFESOR

¿Cualquier modelo puede ser profesor? — **SÍ, si cumple tres**

mejor	que el estudiante en SU tarea
llamable	miles de veces (cabe o se paga)
con licencia	que permita entrenar con sus salidas

¿Se sabe de qué modelo se destiló? — **NO, de forma confiable**

huellas de estilo	indicio, no prueba
marca de agua	solo si el profesor la pone
se audita el ACCESO	no el texto

¿Cómo no incumplir las políticas? — **DOS documentos**

licencia del modelo	pestaña License en HF
términos de la API	OpenAI/Anthropic/Google: no entrenar competidores
en su README	profesor · licencia · uso permitido

Qwen (Apache 2.0) y Llama 3 sí sirven de profesor para su proyecto

T: un dial sobre la ruleta. Profesor: mejor, llamable, con licencia. Lo que se audita es el acceso, no el texto.

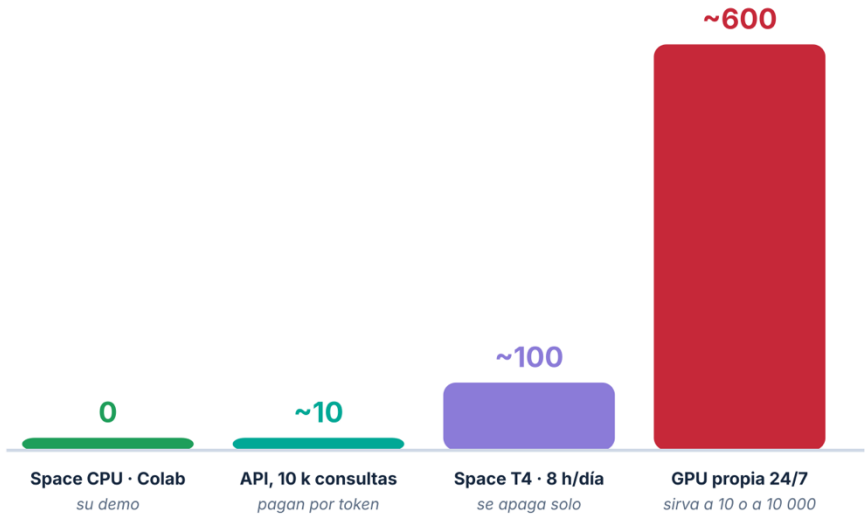
temperatura 0 y semilla fija para comparar; T alta solo cuando la variedad es el producto

Costos, las reglas de M5, y DevOps

"costos :(" · "las reglas de despliegue para M5" · "¿qué buenas prácticas de DevOps traer?"

1 · ¿CUÁNTO CUESTA TENER EL MODELO ENCENDIDO?

USD por mes, orden de magnitud



la cuenta de servilleta

peticiones × tokens × precio/token

o, con GPU propia: horas encendido × precio/hora

poco uso → API · mucho uso o datos sensibles → GPU propia

2 · LAS 7 REGLAS DE M5

lo que tiene que ser cierto el 5 NOV

1. **URL pública** — *que abre desde otro PC*
2. **sistema completo** — *M1 + M2 + M3 + M4, no un mock*
3. **una eficiencia medida** — *tabla antes / después*
4. **health check** — *"ok" + versión*
5. **logs JSON** — *una línea por petición*
6. **W&B** — *latencia y uso en un dashboard*
7. **repo + reporte** — *corre desde cero*

3 · DEVOPS → MLOPS

la misma práctica, el artefacto cambia

git	modelo + eval set versionados
tests en CI	el harness en cada push
Docker	requirements == el Space
canary	10 % del tráfico al estudiante
monitoreo	p95 errores · drift
secretos	API keys fuera del código

Costo = tokens × precio (o horas × precio). M5 = 7 reglas. MLOps = DevOps con el harness como suite de tests.

una GPU olvidada encendida es la factura del mes: apagado automático, siempre

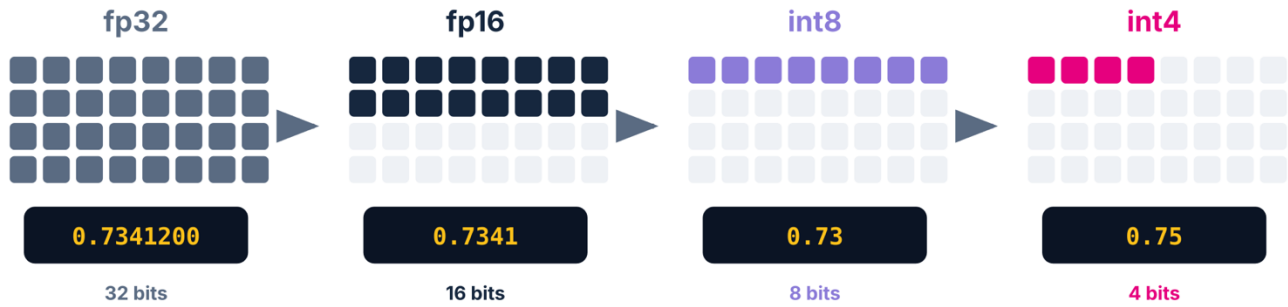
Menos bits, mismo modelo.

La palanca gratis: no se reentrena nada. Cada peso se redondea a menos bits y el modelo ocupa 4× menos. Lo que se pierde, se mide.

Cuantizar: los mismos pesos, con menos bits

como pasar una foto de 24 bits a 8: pesa menos y casi nadie nota — el modelo no se reentrena, se redondea

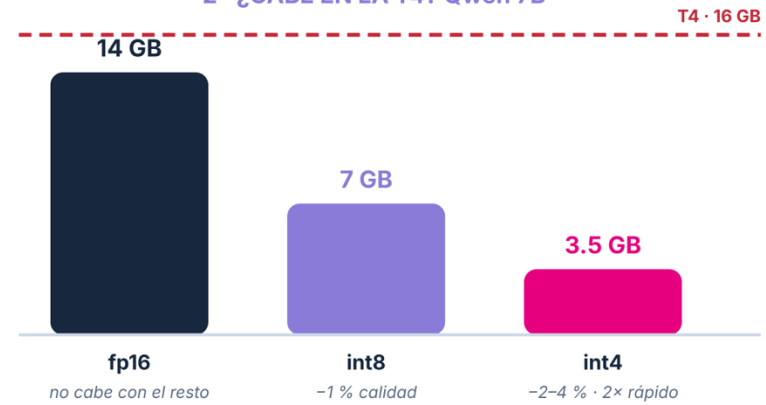
1 · UN PESO ES UN NÚMERO. ¿CUÁNTOS BITS NECESITA?



el peso era 0.73412 → en int4 queda 0.75. Error: 0.016. Sobre millones de pesos, casi se cancela.

no se reentrena nada: se redondea

2 · ¿CABE EN LA T4? Qwen 7B



3 · CON QUÉ (sin reentrenar)

bitsandbytes
al cargar · una línea
`load_in_4bit=True`
hoy en el lab

GPTQ / AWQ
con calibración · mejor calidad
`ya cuantizado en HF`
para servir con vLLM

GGUF
para CPU · llama.cpp / Ollama
`corre en un portátil`
para un Space en CPU

4 · SE COMBINAN (y cada paso cede algo)

7B fp16	14 GB · 6.4 s · 0.78
+ int4	3.5 GB · 3.1 s · 0.76
destilar 1.5B	3 GB · 1.4 s · 0.71
ambos	0.8 GB · 0.9 s · 0.69

memoria · latencia · calidad (harness). Números ilustrativos: los suyos salen del notebook.

Regla de orden: primero cuantizar (gratis) → si no alcanza, destilar (horas) → si aún no, un modelo base más chico.

Cuantizar = redondear los pesos a menos bits: 4× menos memoria sin reentrenar, ~1–4 % de calidad. Primero cuantizar, luego destilar.

se mide igual que todo: calidad con el harness, latencia p50/p95, memoria — con temperatura 0 y semilla fija

El mismo Qwen, la misma pregunta, dos cargas.

LA CARGA

Dos líneas de diferencia

```
# fp16 (no cabe en T4 con el resto)
m = AutoModelForCausalLM.from_pretrained(
    ID, torch_dtype=torch.float16,
    device_map="auto")

# int4 (bitsandbytes, al vuelo)
bnb = BitsAndBytesConfig(load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_quant_type="nf4")
m = AutoModelForCausalLM.from_pretrained(
    ID, quantization_config=bnb,
    device_map="auto")
```

LO QUE MIRAMOS

Tres números y una lectura

Memoria en GPU: torch.cuda.memory_allocated() antes y después de cargar.

Latencia: la misma pregunta 5 veces, p50 y p95 (ignoren la primera: es el calentamiento).

Calidad: 10 casos de su eval set por el harness, con la misma semilla y temperatura 0.

Lectura: ¿cuánto se perdió? ¿vale lo que se ganó? Eso lo decide el contrato, no la intuición.

La trampa: comparar con temperatura > 0. Dos respuestas distintas no significan que el modelo cuantizado sea peor; significan que sortearon distinto. Temperatura 0 y semilla fija para comparar.

Cargar una vez, atender muchas.

Un modelo en un notebook atiende a una persona: ustedes. Un modelo servido atiende a cualquiera, a cualquier hora, por una URL. Es la diferencia entre un experimento y un sistema.

Cuatro formas de servir, de la más simple a la de producción.

HERRAMIENTA	QUÉ ES	CUÁNDO	LO QUE LES DA
Gradio (en Colab o en un Space)	una interfaz web sobre una función Python	su demo del 5 de noviembre	URL pública, UI, cola de peticiones básica
FastAPI + Uvicorn	una API REST propia: POST /responder	cuando otro sistema (no una persona) llama al modelo	JSON entra, JSON sale; el contrato para integrar
vLLM	un servidor especializado en LLMs	muchos usuarios a la vez; latencia importa	continuous batching + KV cache paginado: 5–20× más throughput
llama.cpp / Ollama	servir modelos GGUF en CPU	sin GPU; portátil, edge, demo en CPU gratis	un modelo 7B int4 en una laptop a 10–20 tokens/s

- **p50 y p95** — la mediana dice cómo le va a la mitad de los usuarios; el p95 dice cómo le va al que se queja. Reporten las dos.
- **Tokens por segundo** — la métrica del generador: 20 t/s se lee cómodo; 5 t/s desespera. Streaming disimula hasta cierto punto.

Para la demo basta Gradio. vLLM y FastAPI son lo que verían en una empresa; quedan como profundización.

De Colab
a una URL.

app.py, requirements.txt, un Space. Y encima: health check, logs estructurados y W&B. Eso es "producción verificable": 4 de los 30 puntos de la demo.

De Colab a una URL: el despliegue instrumentado, paso a paso

lo que hacen hoy en el lab, y lo que tiene que estar vivo el 5 de noviembre: app.py + requirements + Space + health + logs + W&B

1 · app.py

una función + Gradio

```
def responder(pregunta, imagen):

return sistema_multimodal(...)

gr.Interface(responder, ...).launch()
```

carga el modelo UNA vez, arriba del archivo

2 · requirements.txt

versiones fijadas

```
transformers==4.45.2

gradio==4.44.0

bitsandbytes, peft, chromadb…
```

lo que corrió en Colab, con ==; sin esto, el Space falla al mes

3 · el Space

git push y listo

```
huggingface.co/new-space

SDK: Gradio · hardware: CPU o T4

git push → build → URL pública
```

secretos (API keys) en Settings → Secrets, nunca en el código

4 · health + logs

para que sea verificable

```
@app.get("/health") → {"ok", "v1.2"}

log.info(json.dumps({ts, q, lat, tool}))

una línea JSON por petición
```

lo primero que muestran en la demo: "está vivo, y esto registró"

5 · W&B

observar en producción

```
wandb.init(project="demo-eafit")

wandb.log({"latencia": t, "tool": ...})

un dashboard con uso y p95
```

es la evidencia de "producción verificable": 4 de 30 puntos

Lo que pasa cuando alguien abre la URL (y por qué el primer arranque es lento):

1. **el Space arranca**
   instala requirements, corre app.py, carga el modelo: 2–5 min, UNA vez
2. **queda esperando**
   el modelo en memoria; si nadie lo usa 48 h, se "duerme" (CPU gratis) y vuelve a arrancar
3. **llega una petición**
   Gradio la recibe → responder() → el modelo ya cargado responde en segundos
4. **se registra**
   el log escribe la línea JSON; W&B recibe la métrica; /health sigue diciendo ok
5. **llega otra**
   misma memoria, mismo modelo: la cocina ya está prendida (duda 1 del recap)

Lo que prueban en vivo el 5 de noviembre, en este orden:

- [ ] abrir la URL desde el computador del aula (no el suyo): carga
- [ ] /health responde ok con la versión del modelo
- [ ] un caso fácil → respuesta en segundos; muestran la línea del log
- [ ] un caso con imagen o con tool → se ve la tool en el log
- [ ] el caso adversarial → el sistema se niega (la válvula)
- [ ] el dashboard de W&B con las peticiones de hoy y la latencia p95
- [ ] la tabla antes/después de la eficiencia (destilado o cuantizado)

Si el Space se cae en vivo: capturas de respaldo de cada punto. Pero la demo es en vivo.

app.py + requirements + Space + health + logs + W&B. El modelo se carga una vez; la URL atiende muchas. Eso es "producción verificable".

hoy terminan con la URL funcionando; esta semana la pulen; el 5 de noviembre la abren desde otro computador

Lo que convierte "funciona en mi máquina" en "se puede verificar".

HEALTH CHECK

¿Está vivo? ¿Qué versión?

```
# en Gradio: una pestaña "Estado"
def health():
    return {"status": "ok",
            "modelo": MODEL_ID,
            "version": "1.2",
            "cuantizado": "int4",
            "uptime_s": time.time() - T0}

# en FastAPI: GET /health
# lo primero que abren en la demo
```

LOGGING ESTRUCTURADO

Una línea JSON por petición

```
log.info(json.dumps({
  "ts": time.time(),
  "pregunta": anonimizar(q),
  "tool": tool_usada,
  "latencia_ms": int(dt * 1000),
  "tokens": n_tokens,
  "version": "1.2"}))

# JSON por línea = se puede filtrar,
# contar y graficar. Texto libre, no.
```

Con esos logs responden en la demo: "hoy atendimos 37 peticiones, p95 de 2.8 s, la tool de visión se llamó 9 veces, ninguna falló". Sin logs, solo pueden decir "creemos que funciona".

Al notebook · Desplegar instrumentado

De su pipeline a una URL con health, logs y W&B

Parte A: cuantizan su generador a int4 y llenan la tabla antes/después (si en S14 destilaron, la comparación es profesor vs. estudiante vs. estudiante int4). Parte B: escriben app.py con Gradio sobre sistema_multimodal, con pestaña de estado y logging JSON; lo prueban en Colab con share=True. Parte C: lo suben a un Space de HF y conectan W&B; abren la URL desde otro dispositivo y ven la primera métrica llegar.

```
int4 + tabla → app.py (Gradio + health + log) → share=True → Space de HF → wandb.log → URL desde el celular
```

- S15_Lab_Despliegue.ipynb

Clase grabada: pausen el video y trabajen el lab con su equipo (~50 min). Vuelvan para el cierre.

Lo que tiene que existir ahora.

- **Una URL que abre desde el celular** — no desde su Colab: desde otro dispositivo, con otra red. Si abre ahí, abre el 5 de noviembre.
- **La pestaña de estado responde** — con el modelo, la versión y si está cuantizado.
- **Los logs se están escribiendo** — una línea JSON por petición; ya tienen las primeras cinco.
- **W&B recibió algo** — al menos la latencia de las peticiones de hoy. El dashboard, aunque feo, existe.
- **La tabla antes/después de la eficiencia** — cuantizado o destilado, con calidad, latencia y memoria.

Si todo eso es cierto, M5 está al 90 %. Lo que falta para el jueves: el reporte, pulir el README y ensayar los 15 minutos con reloj.

Lo que no se mide
no se puede defender.

Un sistema en producción cambia: las preguntas cambian, los datos cambian, el modelo no. Observar es saber cuándo dejó de funcionar antes de que el usuario lo diga.

Cuatro gráficas que caben en un dashboard y responden a la rúbrica.

Latencia por petición (p50, p95 en ventana)
wandb.log({"latencia_ms": dt}). Si el p95 sube un día, algo cambió: más usuarios, un Space dormido, un modelo más lento.

Uso: peticiones por hora y por tool
cuántas, cuándo, cuáles llamaron la tool de visión o el RAG. Es el "quién lo usa y para qué" de la demo.

Calidad en producción: el harness sobre una muestra
cada N peticiones, el juez (medido, S14) califica una muestra y se loguea el promedio. Si baja, es drift: las preguntas ya no son las del eval set.

Errores y negativas
cuántas veces el sistema devolvió error, y cuántas se negó (la válvula). Una negativa no es un error: es el sistema funcionando.

En la rúbrica, "producción verificable" (4 pts) = URL + health + logs + este dashboard. Y "rigor de evaluación" (5 pts) sube si muestran calidad en producción, no solo en el eval set.

Demo day · 5 NOV · 15 min por equipo · 30 % · nota individual.

1

EL REPORTE FINAL
Problema y usuario · decisiones por módulo · scorecard final vs. baselines M1 → M4 · ablation · análisis de errores · límites y ética (licencia del profesor, sesgos, válvula, datos).

2

EL REPO LIMPIO
README que permita correrlo desde cero; m1/ a m5/ con lo que ya entregaron; la URL del Space arriba del todo.

3

EL ENSAYO CON RELOJ
Los 15 minutos completos, una vez, con los tres casos elegidos y la URL abierta desde otro computador. Cada integrante explica cualquier módulo.

4

LAS CAPTURAS DE RESPALDO
De la URL, del health, de los logs, del dashboard y de la tabla. Por si el Space se cae en vivo. Pero la demo es en vivo.

La infografía del demo day está en Teams: qué decir minuto a minuto y cómo visibilizar un resultado técnico. Cada número con su baseline.

Tres ideas más de la misma familia.

Speculative decoding

El estudiante adivina 4–5 tokens, el profesor los verifica en una sola pasada. 2–3× más rápido sin cambiar la salida. Destilación y serving trabajando juntos.

Canary y rollback

El modelo nuevo recibe el 10 % del tráfico; se compara su scorecard en producción con el viejo; si gana, 100 %; si pierde, vuelve en un clic. DevOps aplicado a modelos.

Guardrails en producción

Una capa antes y después del modelo: filtros de entrada (inyección de prompt, datos personales) y de salida (formato, negativas). Es la válvula de escape, industrializada.

Fuera del alcance de M5. Pero si en la demo alguien pregunta "¿y cómo lo actualizarían sin romperlo?", canary es la respuesta.

Lo que construyeron, en el orden en que lo construyeron.

PARADA	LA PREGUNTA	LO QUE RESPONDIERON	EN SU SISTEMA HOY
M1 · S02–S04	¿qué modelo, y si no habla mi dominio?	familias del transformer; fine-tuning con LoRA	el generador adaptado
M2 · S05–S06	¿es bueno de verdad?	harness de 3 dimensiones, eval set, juez	la vara que sigue midiendo todo
M3 · S07–S10	¿y lo que no se puede fine-tunear?	RAG ingenuo → avanzado → tools y agente; RAGAS	el retriever, las tools, el bucle
M4 · S11–S13	¿y si necesita ver?	ViT, CLIP, VLM, difusión; un componente integrado y medido	los ojos del sistema
M5 · S14–S15	¿sirve fuera del notebook?	destilar, cuantizar, servir, desplegar, observar	la URL que abren el jueves

Cinco preguntas. Cada una con su tabla. Ese es el método, y es el mismo para lo que venga después del curso.

SI4006 · EL CURSO EN UN CUBO

M1 M2 M3 M4 M5

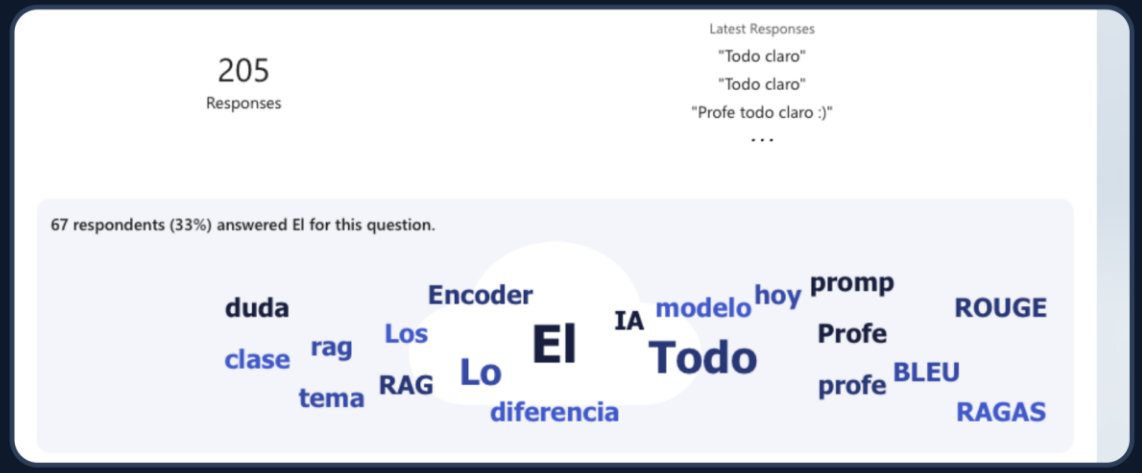
Agosto: un cubo.

Un modelo base. Sabe de todo, no sabe de ustedes.

baseline



SI4006 · 104 DÍAS · 16 SESIONES · 205 RESPUESTAS AL QR



pregunta 1 del QR, todo el semestre: "¿qué fue lo que menos entendiste hoy?"

205 respuestas **104** días

a 1:11 por respuesta: cuatro horas de su tiempo diciéndome qué no entendieron. Gracias por eso.

Miren su propia nube.

En agosto escribían:

- Encoder
- RAG
- BLEU
- ROUGE
- diferencia
- duda

Y la palabra más grande, al final, es:

Todo ← *"todo claro": 1 de cada 3 respuestas.*

Eso no lo hizo el curso. Lo hicieron ustedes, un jueves a la vez.
Esa nube es su scorecard. Y el delta contra el baseline de agosto… está a la vista.

Hace 104 días, "RAG" era una duda. Hoy es una fila de su scorecard, desplegada en una URL.

Lo que viene no es más difícil que eso. Es exactamente igual de difícil: una duda, una tabla, una URL. Repitan.

Nos vemos el 5 de noviembre. Vayan a brillar.

Una duda, una tabla, una URL.
Repitan.

Cuantizar sin reentrenar; servir cargando una vez; desplegar con health, logs y W&B. El sistema salió del notebook. Nos vemos el jueves 5 de noviembre: vayan a brillar.